МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«САНКТ-ПЕТЕРБУРГСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ ПЕТРА ВЕЛИКОГО»

Институт компьютерных наук и технологий

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 «Математика и компьютерные науки»

Отчёт о выполнении курсовой работы по дисциплине Теоретические основы баз данных Автоматизация работы библиотеки.

Студент,
группа 5130201/30002 _____ Сафин А. Р.

Преподаватель _____ Попов С. Г.

«20» 05 2025г.

1/3 25.03.25

2/3 20.05.25

Санкт-Петербург

2025

Реферат

Предметной областью курсовой работы является библиотека. Проблемой является отсутствие систематизированного хранения информации о читателях, сотрудниках библиотеки, экземплярах и операциях выдачи и возврата, что усложняет процесс управления библиотекой. В рамках работы была разработана база данных, которая хранит информацию о сущностях библиотеки и их связях. Были сформированы и выполнены 8 запросов, 1 из которых включает в себя второй вариант.

Во время выполнения работы использовались диалект PostgreSql, DBeaver 24.3.0, командная строка, Visual Studio версии 17.8.3, Python версии 3.13.

Полученная в результате работы база данных может быть использована в библиотеках для автоматизации учета читателей, книжных экземпляров, операций выдачи и возврата.

Содержание

Реферат **2**

Введение **4**

1 Постановка задачи **5**

2 Аналитика работы библиотеки **6**

2.1 Описание предметной области 6
2.2 Цели функционирования 8
2.3 Описание сущностей 8
2.4 Описание ролей 9
2.5 ER-диаграмма 9
2.6 Схема связи объектов 11

3 Проектирование **12**

3.1 Лингвистическое описание 12
3.2 Атрибуты таблиц 13

4 Программирование **19**

4.1 Генерация записей 19
4.2 Запросы к базе данных 20
4.2.1 Запрос 1 20
4.2.2 Запрос 2 22
4.2.3 Запрос 3 23
4.2.4 Запрос 4 25
4.2.5 Запрос 5 27
4.2.6 Запрос 6 29
4.2.7 Запрос 7 30
4.2.8 Запрос 8 31

Заключение **34**

Использованные источники **35**

Приложение A: Создание базы данных **36**

Приложение B: Заполнение базы данных **41**

Введение

База данных (БД) — логически структурированная совокупность постоянно хранимых данных, характеризующих актуальное состояние некоторой предметной области и используемых прикладными программными системами какого-либо предприятия.

Система управления базами данных (СУБД) - комплекс программных и языковых средств, предназначенный для модификации и извлечения из БД необходимых пользователю данных, а также для создания баз данных, поддержания их в работоспособном состоянии, обеспечения безопасности баз данных и решения других задач администрирования.

Разработка базы данных – это процесс проектирования, создания и на- стройки структуры для хранения, управления и обработки данных. Она необ- ходима для организации больших объемов информации, чтобы данные были легко доступны, быстро обрабатывались и обеспечивали структурированное взаимодействие между пользователями и приложениями. Базы данных осо- бенно важны для автоматизации процессов, аналитики и принятия решений, так как позволяют эффективно хранить, изменять, искать и анализировать сведения. Среди преимуществ разработки базы данных: упрощение управле- ния данными, минимизация ошибок, повышение скорости работы с информа- цией, улучшение безопасности данных, а также возможности масштабирова- ния и совместной работы в реальном времени. Корректно спроектированная база данных помогает упорядочить информацию и сократить затраты на её обработку.

1 Постановка задачи

Целью данной работы являетя реализация функционирующей реляционной базы данных для библиотеки и реализация полученных запросов.

Задачи

1. Проанализировать предметную область.
2. Создать ER-диаграмму в нотации Чена.
3. Определить цели функционирования базы данных.
4. Спроектировать схему базы данных.
5. Реализовать базу данных в PostgreSql.
6. Создать программу заполнения базы данных.
7. Реализовать запросы к базе данных.
8. Представить результаты запросов графически.

2 Аналитика работы библиотеки

2.1 Описание предметной области

Библиотека — это учреждение, собирающее и хранящее произведения печати и письменности для общественного пользования, а также осуществляющее справочно-библиографическую работу. Она располагает организованным фондом книг и предоставляет эти книги во временное пользование.

Деятельность библиотек по обслуживанию читателей осуществляется в двух формах. В первом случае читатель может получить книгу из библиотеки в своё полное распоряжение на определённый срок. В основном книги предоставляются бесплатно, но на некоторые книги, имеющиеся в небольшом количесиве экземпляров, библиотека может назначать арендную плату или денежный залог. Во втором случае читатель может знакомиться с книгой в специальном зале библиотеки, называемом читальным залом. Такие залы обычно оборудованы книжными стеллажами, столами и стульями. Могут присутствовать столы, оборудованные компьютерами для работы с информацией на электронных носителях, или круглые столы, для коллективной работы. Также в читальном зале могут быть витрины для проведения выставок.

Читатель - это человек, обладающий специальным документом, читательским билетом, который подтверждает его право на посещение библиотеки и пользование библиотечными изданиями. Читательский билет оформляется и выдаётся библиотекарем при первом посещении библиотеки. Для получения билета читатель ригистрируется в картотеке, содержащей следующие анкетные данные: фамилия, имя, отчество, серия и номер паспорта, номер телефона и адрес проживания.

Библиотекари - сотрудники библиотеки, задача которых заключается в каталогизации и классификации книг и обслуживании читателей. Библиотекари помогают искать книжные издания в библиотеке и консультируют читателей по поводу выбора литературы. осуществляют регистрацию новых читателей, выдачу и возврат книг. Также они осуществляют регистрацию новых читателей, выдачу и возврат книг. При каждой выдаче библиотекарь записывает в картотеку экземпляр, номер читательского билета, на который оформлена эта выдача, дату и время выдачи. При возврате книги библиотекарь вносит в картотеку пометку о возврате и проверяет не был ли этот возврат просрочен.

Администраторы - сотрудники, которые ведут учет поступления новых изданий в библиотеку, а также следят за состоянием книжного фонда. Они осуществляют добавление информации в картотеку при поступлении новых книг, обновление информации по книгам и издательствам, а также формирование и выгрузку финансовых отчётов для бухгалтерии.

Для огранизации книг внутри библиотеки и упрощения их поиска, при поступ-

лении каждой книги в фонд для неё учитываютя следующие данные: название, ФИО всех авторов, ISBN - специальный номер, позволяющий уникально идентифицировать каждое издание книги, жанр, год издания, издательство и код УДК, позволяющий классификацировать содержащуюся в книге информацию. Внутри библиотечных залов книги располагаются на специальных книжных стеллажах. Общее направление расположения книг на стеллажах - сверху вниз, а на каждой из полок - слева направо. Порядок расстановки основывается на содержательных признаках книги, таких как отрасль знания, тема или жанр книги, или на основе формальных признаков, как алфавитный порядок имён авторов и заглавий книги, времени издания или места издания.

Читатели берут книги на время, читают их в библиотеке или выносят на дом. Библиотека отслеживает количество выданных книг и интерес людей к конкретным темам и книгам. После прочтения читателем книга возвращается в библиотеку, где она проверяется на наличие повреждений. Если их нет, книга снова становится доступной для других читателей. В случае если книга повреждена, устарела или не востребована, она может быть списана из коллекции. При этом книгу утилизируют или передают в фонд другой библиотеки. При этом некоторые книги могут быть отремонтированы или восстановлены для продления их срока службы, особенно если они имеют историческую или культурную ценность.

2.2 Цели функционирования

1. Учёт всех экземпляров книг.
2. Учёт всех сотрудников библиотеки.
3. Учёт всех читателей.
4. Учёт всех процессов выдачи, возврата, поступления и списания книг.

2.3 Описание сущностей

1. Издательство — это учреждение, занимающееся изданием книг. Атрибуты: Название, телефон, адрес.
2. Экземпляр - отдельная книга в библиотеке, которой присвоен уникальный полочный номер. Атрибуты: входные данные книги, администратор, время поступления, время списания.
3. Выдача - это процесс, когда читателю предоставляют во временное пользование книжное издания из библиотечного фонда. Атрибуты: библиотекарь, экземпляр, читатель, время операции.
4. Возврат - это процесс, при котором читатель возвращает взятую из библиотеки книгу обратно в книжный фонд. Атрибуты: библиотекарь, экземпляр, читатель, время операции.
5. Входные данные книги - это набор сведений, характеризующих печатное издание и необходимых для его идентификации, библиотечной классификации, статистического учёта. Атрибуты: Название, ISBN, инлекс УДК, издательство, автор, дата издания.
6. Поступление - это процесс, в ходе которого новые книжные издания добавляются в книжный фонд библиотеки. Атрибуты: Администратор, дата, экземпляр.
7. Списание - это процесс исключения книжных изданий из библиотечного фонда, в результате которого они утилизируются или передаются в фонд других библиотек. Атрибуты: Администратор, дата, экземпляр.

2.4 Описание ролей

1. Читатель - человек, обладающий читательским билетом. Читатель может брать книги из библиотеки в своё полное пользование или читать их в читальном зале библиотеке.
   Атрибуты: Имя, фамилия, отчество, телефон, серия и номер паспорта, адрес, дата регистрации, библиотекарь, осуществлявший регистрацию.

2. Библиотекарь - сотрудники библиотеки. Библиотекарь регистрирует читателей, оформляет выдачу экземпляров изданий читателю, оформляет возврат экземпляров от читателя.
   Атрибуты: Имя, фамилия, отчество, телефон, серия и номер паспорта.

3. Администратор - сотрудник библиотеки. Администратор оформляет поступление экземпляров изданий, оформляет списание экземпляров изданий.
   Атрибуты: Имя, фамилия, отчество, телефон, серия и номер паспорта.

2.5 ER-диаграмма

Чтение ER-диаграммы, изображённой на рисунке 1.

1. Библиотекарь оформляет операцию выдачи или возврата экземпляра книги на читателя.

2. Тип операции характеризует операцию.

3. Библиотекарь регистрирует читателя.

4. Читатель читает экземпляр книги.

5. Администратор оформляет поступление экземпляра книги.

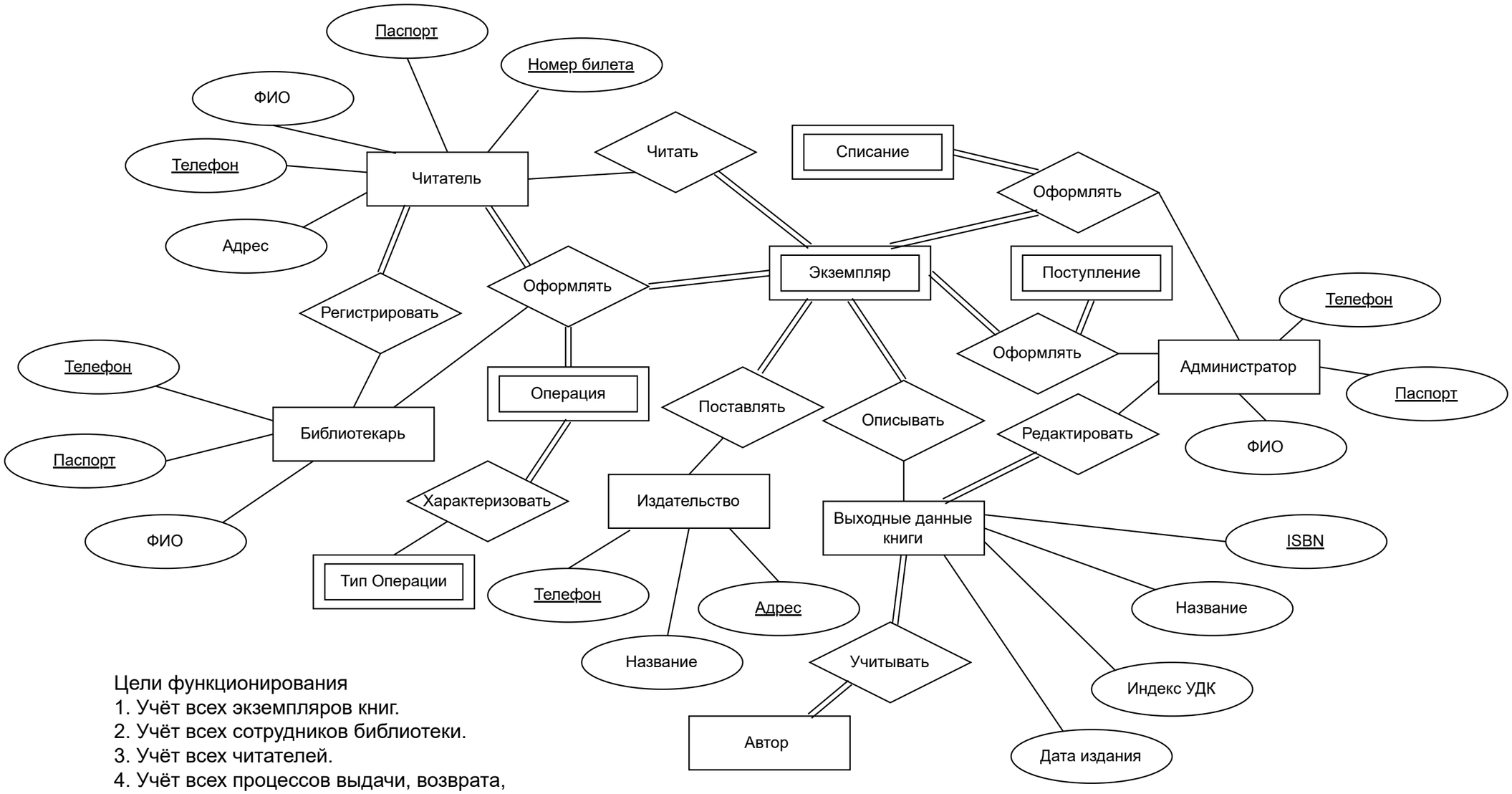
6. Администратор оформляет списание экземпляра книги.

7. Издательство поставляет экземпляр.

8. Администратор редактирует входные данные книги.

9. Входные данные описывают экземпляр книги.

10. Входные данные учитывают автора книги.



Цели функционирования
1. Учёт всех экземпляров книг.
2. Учёт всех сотрудников библиотеки.
3. Учёт всех читателей.
4. Учёт всех процессов выдачи, возврата, поступления и списания книг.

Рис. 1. ER-диаграмма

2.6 Схема связи объектов

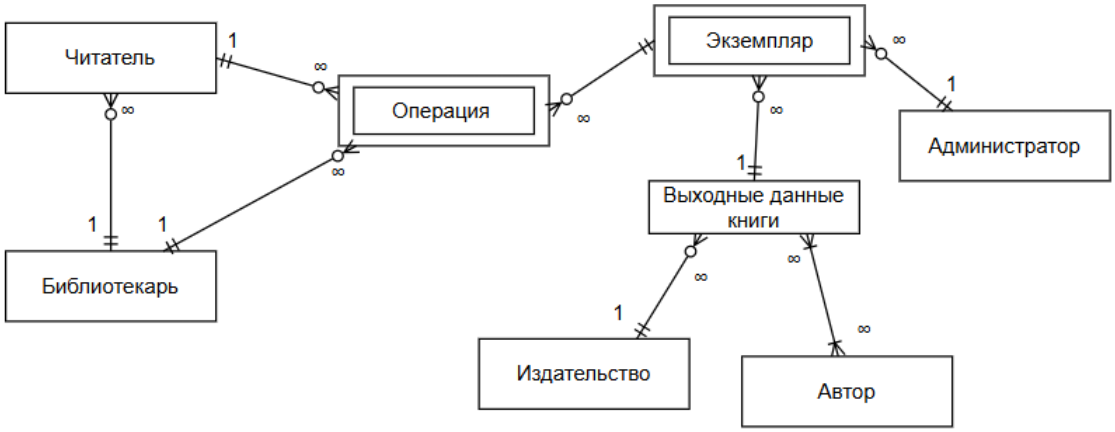


Рис. 2. Схема объектов.

3 Проектирование

3.1 Лингвистическое описание

1. Reader(<id, serial>, <f_name, varchar(1478)>, <m_name, varchar(14)>, <l_name, varchar(700)>, <pass_ser, int4>, <pass_num, int4>, <phone, varchar(12)>, <address, varchar(512)>, <reg_date, date>, <libr_id, int4>)

2. Librarian(<id, serial>, <f_name, varchar(1478)>, <m_name, varchar(14)>, <l_name, varchar(700)>, <pass_ser, int4>, <pass_num, int4>, <phone, varchar(12)>)

3. Administrator(<id, serial>, <f_name, varchar(1478)>, <m_name, varchar(14)>, <l_name, varchar(700)>, <pass_ser, int4>, <pass_num, int4>, <phone, varchar(12)>)

4. Book(<id, serial>, <ISBN, char(17)>, <name, varchar(65535)>, <publish_date, date>, <UDK, varchar(15)>, <publish_id, int4>)

5. Exemplar(<id, serial>, <book_id, int4>, <admin_id, int4>, <receipt, timestamp>, <write_off, timestamp>, <name, varchar(50)>, <phone, varchar(12)>, <address, varchar(512)>)

6. Publishing(<id, serial>, <name, varchar(255)>, <phone, varchar(12)>, <address, varchar(512)>)

7. Operation(<id, serial>, <reader_id, int4>, <libr_id, int4>, <exemplar_id, int4>, <op_date, timestamp>, <type, varchar(7)>)

8. Author(<id, serial>, <f_name, varchar(1478)>, <m_name, varchar(14)>, <l_name, varchar(700)>)

9. Authorship(<id, serial>, <book_id, int4>, <auth_id, int4>)

3.2 Атрибуты таблиц

В таблицах представлены атрибуты каждой таблицы проектируемой базы данных.

Читатель							
Номер атрибута	Название (Русский)	Название(Английский)	Тип данных	Значение по умолчанию	Признак ключа	Ограничение	Обоснование ограничения
1	Код читателя	id	serial		Первичный ключ	Not null, уникальный	Первичный ключ (суррогатный)
2	Имя	f_name	varchar(1478)			Not null, неуникальный	Самое большое имя содержит 1478 символов, имена могут повторяться
3	Фамилия	m_name	varchar(700)			Not null, неуникальное	Самая большая фамилия содержит 700 символов, фамилии могут повторяться
4	Отчество	l_name	varchar(14)			Null, неуникальное	Самое большое отчество содержит 14 символов, отчество могут повторяться, у человека может не быть отчества
5	Серия паспорта	pass_ser	integer			Not null, не уникальное, число из 4 цифр	Составной альтернативный ключ, серия и номер паспорта уникальны для каждого человека
6	Номер паспорта	pass_num	integer			Not null, не уникальноечисло из число из 6 цифр	Составной альтернативный ключ, серия и номер паспорта уникальны для каждого человека
7	Телефон	phone	char(12)			Not null, уникальное, может содержать только цифры и символ +, должен соответствовать шаблону: +dddddddddddd	Телефон уникален для каждого читателя. Запись телефона в международном формате.
8	Адрес проживания	adress	varchar(512)			Not null, не уникальное	Два читателя могут жить по одному адресу
9	Код библиотекаря	libr_id	integer		Внешний ключ на библиотекаря (id)		
10	Дата регистрации	reg_date	date			Not null, меньше текущей даты	Дата регистрации должна быть меньше текущей даты

Таблица 1. Читатель.

Библиотекарь							
Номер атрибута	Название	Название(Английский)	Тип данных	Значение по умолчанию	Признак ключа	Ограничение	Обоснование ограничения
1	Код библиотекаря	id	serial		Первичный ключ (суррогатный)	Not null, уникальный	Первичный ключ (суррогатный)
2	Фамилия	f_name	varchar(1478)			Not null, неуникальный	Самое большое имя содержит 1478 символов, имена могут повторяться
3	Имя	m_name	varchar(700)			Not null, неуникальное	Самая большая фамилия содержит 700 символов, фамилии могут повторяться
4	Отчество	l_name	varchar(14)			Null, неуникальное	Самое большое отчество содержит 14 символов, отчество могут повторяться, у человека может не быть отчества
5	Серия паспорта	pass_ser	integer			Not null, не уникальное, число из 4 цифр	Составной альтернативный ключ, серия и номер паспорта уникальны для каждого человека
6	Номер паспорта	pass_num	integer			Not null, не уникальноечисло из число из 6 цифр	Составной альтернативный ключ, серия и номер паспорта уникальны для каждого человека
7	Телефон	phone	varchar(12)			Not null, уникальное, может содержать только цифры и символ +, должен соответствовать шаблону: +dddddddddddd	Телефон уникален для каждого библиотекаря. Запись телефона в международном формате.

Таблица 2. Библиотекарь.

Администратор							
Номер атрибута	Название	Название(Английский)	Тип данных	Значение по умолчанию	Признак ключа	Ограничение	Обоснование ограничения
1	Код администратора	id	serial		Первичный ключ (суррогатный)	Not null, уникальный	Первичный ключ (суррогатный)
2	Фамилия	f_name	varchar(1478)			Not null, неуникальный	Самое большое имя содержит 1478 символов, имена могут повторяться
3	Имя	m_name	varchar(700)			Not null, неуникальное	Самая большая фамилия содержит 700 символов, фамилии могут повторяться
4	Отчество	l_name	varchar(14)			Null, неуникальное	Самое большое отчество содержит 14 символов, отчество могут повторяться, у человека может не быть отчества
5	Серия паспорта	pass_ser	integer			Not null, не уникальное, число из 4 цифр	Составной альтернативный ключ, серия и номер паспорта уникальны для каждого человека
6	Номер паспорта	pass_num	integer			Not null, не уникальноечисло из число из 6 цифр	Составной альтернативный ключ, серия и номер паспорта уникальны для каждого человека
7	Телефон	phone	varchar(12)			Not null, уникальное, может содержать только цифры и символ +, должен соответствовать шаблону: +dddddddddd	Телефон уникален для каждого администратора. Запись телефона в международном формате.

Таблица 3. Администратор.

Книга							
Номер атрибута	Название	Название(Английский)	Тип данных	Значение по умолчанию	Признак ключа	Ограничение	Обоснование ограничения
1	Код книги	id	serial		Первичный ключ (суррогатный)	Not Null, уникальный	Первичный суррогатный ключ
2	ISBN	ISBN	varchar(17)			Not Null, уникальный, шаблон: xxx-*-*****-***-*	Естественный альтернативный ключ, уникальный для каждой книги
3	Название	name	varchar(65535)			Not Null, неуникальный	В названии самой большой книги 27 978 символов, названия книг могут повторяться
4	Дата издания	publish_date	date			Not Null, неуникальный	Разные книги могут быть изданы в один день
5	Код издательства	publish_id	integer		Внешний ключ на издательство (id)	Not null, неуникальный	Внешний ключ на издательство
6	Индекс УДК	UDK	varchar(5)			Not null, неуникальный, шаблон ***.*	Индексы могут повторяться

Таблица 4. Книга.

Экземпляр							
Номер атрибута	Название	Название(Английский)	Тип данных	Значение по умолчанию	Признак ключа	Ограничение	Обоснование ограничения
1	Код экземпляра	id	serial		Первичный ключ (суррогатный)	Not Null, уникальный	Первичный суррогатный ключ
2	Код книги	book_id	integer		Внешний ключ на книгу	Not Null, неуникальный	Внешний ключ на Книгу
3	Код администратора	admin_id	integer		Внешний ключ на администратора (id)	Not Null, неуникальный	Внешний ключ на Администратора
4	Поступление	receipt	timestamp	now()		Not Null, неуникальный, меньше списания или now()	Дата поступления должна быть меньше текущей даты
5	Списание	write_off	timestamp	Null		Null, неуникальный, меньше now(), больше поступления	Дата списания должна быть меньше текущей даты и больше времени поступления

Таблица 5. Экземпляр.

Издательство							
Номер атрибута	Название	Название(Английский)	Тип данных	Значение по умолчанию	Признак ключа	Ограничение	Обоснование ограничения
1	Код издательства	id	serial		Первичный ключ (суррогатный)	Not Null, уникальный	Первичный суррогатный ключ
2	Название	name	varchar(255)			Not Null, неуникальный	Названия могут повторяться
3	Телефон	phone	varchar(12)			Not null, уникальное, может содержать только цифры и символ +, должен соответствовать шаблону: +dddddddddd	Телефон уникален для каждого издательства. Запись телефона в международном формате.
4	Адресс	adress	varchar(512)			Not null, уникальный	Каждое издательство имеет уникальный адрес

Таблица 6. Издательство.

Операция							
Номер атрибута	Название	Название(Английский)	Тип данных	Значение по умолчанию	Признак ключа	Ограничение	Обоснование ограничения
1	Код операции	id	serial		Первичный ключ (суррогатный)	Not Null, уникальный	Первичный суррогатный ключ
2	Тип операции	type	varchar			Not Null, неуникальное	Тип операции: Выдача или Возврат
3	Код экземпляра	exemplar_id	integer		Внешний ключ на экземпляр (id)	Not Null, неуникальный	Внешний ключ на экземпляр
4	Код читателя	reader_id	integer		Внешний ключ на читателя (id)	Not Null, неуникальный	Внешний ключ на читателя
5	Код библиотекаря	libr_id	integer		Внешний ключ на библиотекаря (id)	Not Null, неуникальный	Внешний ключ на библиотекаря
6	Время выдачи	op_date	timestamp	now()		Not Null, неуникальный, меньше now()	Дата выдачи книги должна быть меньше текущей даты

Таблица 7. Операция.

Автор							
Номер атрибута	Название	Название(Английский)	Тип данных	Значение по умолчанию	Признак ключа	Ограничение	Обоснование ограничения
1	Код автора	id	serial		Первичный ключ (суррогатный)	Not Null, уникальный	Первичный суррогатный ключ
2	Имя	f_name	varchar(1478)			Not null, неуникальный	Самое большое имя содержит 1478 символов, имена могут повторяться
3	Отчество	m_name	varchar(700)			Not null, неуникальное	Самая большая фамилия содержит 700 символов, фамилии могут повторяться
4	Фамилия	l_name	varchar(14)			неуникальное	Самое большое отчество содержит 14 символов, отчество могут повторяться, у человека может не быть отчества

Таблица 8. Автор.

Авторство							
Номер атрибута	Название	Название(Английский)	Тип данных	Значение по умолчанию	Признак ключа	Ограничение	Обоснование ограничения
1	Код авторства	id	serial		Первичный ключ (суррогатный)	Not Null, уникальный	Первичный суррогатный ключ
2	Код книги	book_id	integer		Внешний ключ на книгу (id)	Not Null, неуникальный	Внешний ключ на книгу
3	Код автора	auth_id	integer		Внешний ключ на автора (id)	Not Null, неуникальный	Внешний ключ на администратора

Таблица 9. Авторство.

Схема базы данных на русском языке отображена на рисунке 3. Схема базы данных на английском языке отображена на рисунке 4.

Для создания таблиц был написан код SQL. Код приведен в Приложении А.

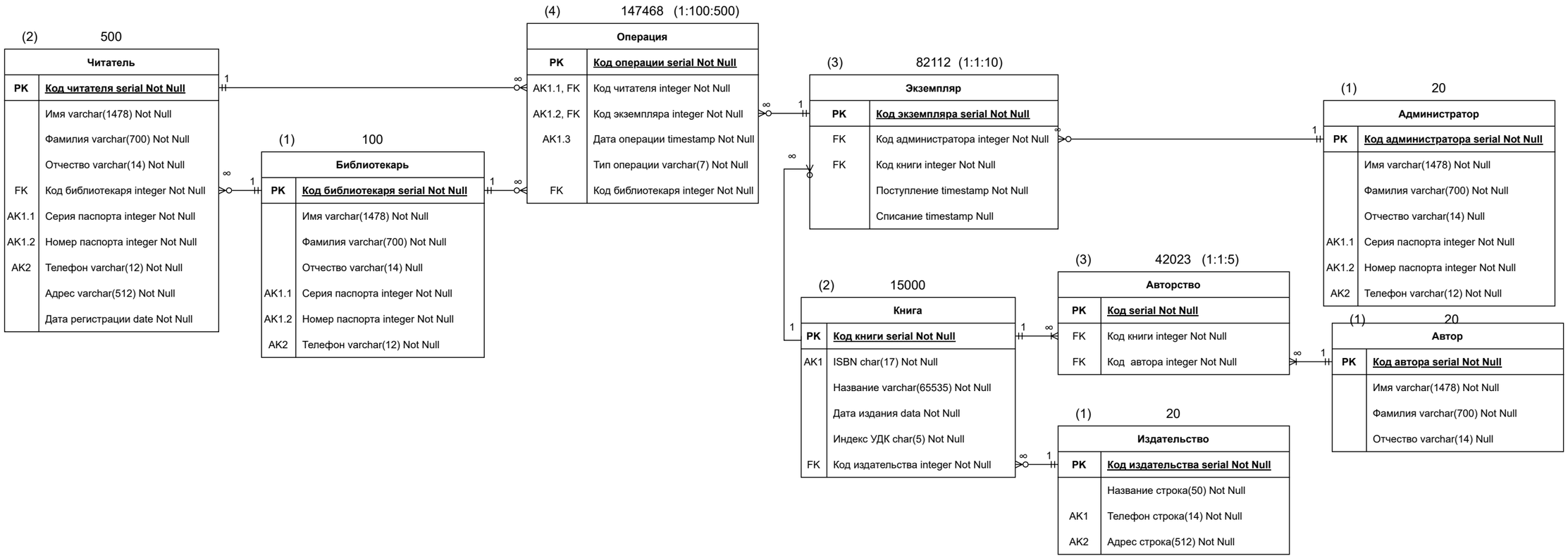


Рис. 3. Схема базы данных на русском языке

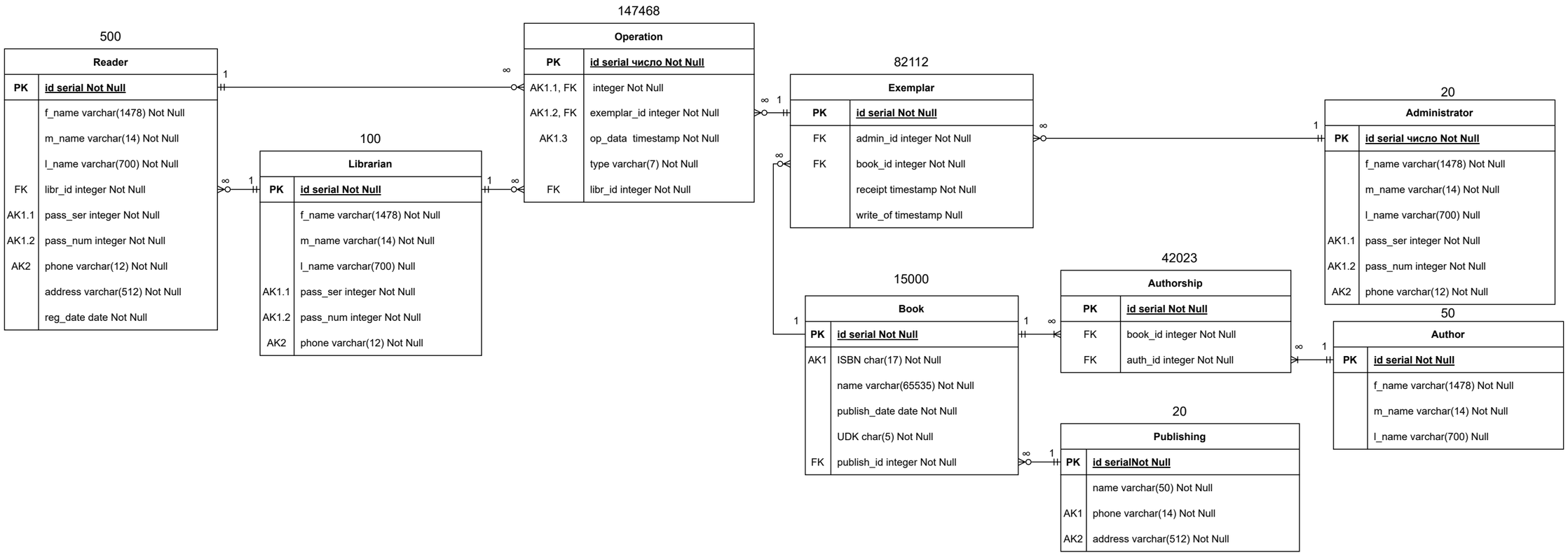


Рис. 4. Схема базы данных на английском языке

Лист проверки выполнения запросов курсовой работы по дисциплине «Теоретические основы баз данных» 4 семестр

Имя, отчество, фамилия Артём Рамидович Сафин Группа 5130201/3000 2

Тема работы Автоматизация работы библиотеки.

№ п.п.	текст запроса	принято запрос	принято отчет
1	1.1 Найти авторов, которых читал читатель А, а экземпляр которых выдавал библиотекарь В 1.2 Не менее двух раз. кто быстрее)	☑ ☑	☐ ☐
2	2.1. Посчитать число библиотекарей, которые выдавали книги экземпляры которых были списаны администратором А	☑	☐
3	3.1 Для каждого издательства посчитать число экземпл.ов книг и число операций, которые с этими экземплярами выполнялись + построить 2D гистограмму	☑	☐
4	Посчитать число книг с одинак. числом экземпляров + построить 2D гистограмму	☑	☐
5	Найти библиотекаря с ~~ма~~ максимальным числом выданных экземпляров и с минимальным числом вып. экземпляров	☑	☐
6	Найти администратора, который списал книг автора А больше чем администратор В.	☑	☐
7	Найти читателей, которые не брали книги издательства А.	☑	☐
8	Для каждого Администратора и каждого издательства подсчитать число списанных экземпляров + построить 3D гистограмму.	☑	☐
9		☐	☐
10		☐	☐

4 Программирование

4.1 Генерация записей

Для заполнения базы данных PostgreSQL тестовыми записями был разработан скрипт на языке Python. Код заполнения представлен в Приложении B.

Для подключения к PostgreSQL и выполнения SQL-запросов использовалась библиотека psycopg2. Подключение к базе данных производилось через функцию create_connection(), использующую параметры конфигурации DB_CONFIG. Для генерации реалистичных тестовых данных, таких как имена, даты или адреса, использовалась библиотека Faker.

Для библиотекарей, администраторов и читателей генерируются данные ФИО с учётом пола, паспортные данные, телефоны и адреса. Для издательств генерируются телефоны и адреса. Для книг генерируются ISBN, UDK, даты изданий. Для каждого экземпляра генерируется дата поступления и в 5 процентах случаев дата списания. Для всех операций случайно определяется тип: выдача или возврат, генерируется время операции.

Для правильного заполнения внешних ключей данные о существующих идентификаторах были получены из таблиц при помощи SQL-команды Select.

Запись данных в таблицы производились с помощью SQL-команды INSERT. При этом данные вставлялись пакетами по 1000 записей для снижения нагрузки на СУБД.

Было заполнено 9 таблиц с общим количеством записей 290293.

Таблица	Количество сгенерированных записей
Читатель	500
Библиотекарь	100
Администратор	20
Автор	50
Издательство	20
Книга	15000
Экземпляр	82112
Операция	147468
Авторство	42023
Все таблицы	290293

Таблица 10. Количество записей в таблицах.

4.2 Запросы к базе данных

4.2.1 Запрос 1

1.1) Найти авторов, которых читал читатель А, и экземпляры которых ему выдавал библиотекарь Б

```
select distinct a.f_name, a.m_name, a.l_name
from "Process"."Operation" as o
inner join "Process"."Exemplar" as e  on e.id = o.exemplar_id
inner join "Publication"."Book" as b on  b.id = e.book_id
inner join "Publication"."Authorship" as ash on ash.book_id = b.id
inner join "Publication"."Author" as a on a.id = ash.auth_id
where o.reader_id = 1 and o.libr_id = 3 and o.type = 'выдача';
```

f_name	m_name	l_name
Валентина	Аркадьевна	Кузьмина
Василиса	Георгиевна	Александрова
Василиса	Леонидовна	Павлова
Дементий	Харламович	Баранов
Кира	Владиславовна	Котова
Лазарь	Филатович	Данилов
Лукьян	Давыдович	Буров
Мартьян	Харламович	Александров
Матвей	Давидович	Сергеев
Прасковья	Петровна	Давыдова
Фёкла	Ниловна	Савина

Рис. 5: Результат запроса 1.1.

Время выполнения: 0:00:00.004819.

Сначала происходит выбор строк из Operation с условиями: reader_id=1, libr_id=3, type='выдача'. Далее последовательно выполняются присоединение таблицы Exemplar по exemplar_id, присоединение таблицы Book по book_id, присоединение таблицы Authorship по book_id и присоединение таблицы Author по auth_id. После этого оператор distinct удаляет повторяющиеся записи авторов. Представление последавательности выполнения запроса отображено на рисунке 6.

```
-> Sort  (cost=25.48..25.49 rows=3 width=48)
   Sort Key: a.f_name, a.m_name, a.l_name
   -> Nested Loop  (cost=1.31..25.46 rows=3 width=48)
      -> Nested Loop  (cost=1.16..24.95 rows=3 width=4)
         Join Filter: (b.id = ash.book_id)
         -> Nested Loop  (cost=0.87..24.56 rows=1 width=8)
            -> Nested Loop  (cost=0.59..24.25 rows=1 width=4)
               -> Index Scan using idx_op_reader_id on "Operation" o  (cost=0.29..15.94 rows=1 width=4)
                  Index Cond: (reader_id = 1)
                  Filter: ((libr_id = 3) AND ((type)::text = 'выдача'::text))
               -> Index Scan using "Exemplar_pkey" on "Exemplar" e  (cost=0.29..8.31 rows=1 width=8)
                  Index Cond: (id = o.exemplar_id)
            -> Index Only Scan using "Book_pkey" on "Book" b  (cost=0.29..0.30 rows=1 width=4)
               Index Cond: (id = e.book_id)
         -> Index Scan using idx_as_book_id on "Authorship" ash  (cost=0.29..0.36 rows=3 width=8)
            Index Cond: (book_id = e.book_id)
      -> Index Scan using "Author_pkey" on "Author" a  (cost=0.15..0.17 rows=1 width=52)
         Index Cond: (id = ash.auth_id)
```

Рис. 6: Explain запроса 1.1.

Реляционная формула:

$$\pi_{\text{distinct } a.f_name, a.m_name, a.l_name}(\sigma_{\text{reader_id}=1 \wedge \text{libr_id}=3 \wedge \text{type}='\text{выдача}'}(\rho_o(Operation))$$
$$\rho_a(Author) \bowtie_{a.id=ash.auth_id} \rho_{ash}(Authorship) \bowtie_{ash.book_id=b.id}$$
$$\rho_b(Book) \bowtie_{b.id=e.book_id} \rho_e(Exemplar) \bowtie_{e.id=o.exemplar_id})$$

1.2) Не менее двух раз.

```
select a.f_name, a.m_name, a.l_name,
count(distinct o.id) as number_of_reads
from "Process"."Operation" as o
inner join "Process"."Exemplar" as e  on e.id = o.exemplar_id
inner join "Publication"."Book" as b on  b.id = e.book_id
inner join "Publication"."Authorship" as ash on ash.book_id = b.id
inner join "Publication"."Author" as a on a.id = ash.auth_id
where o.reader_id = 1 and o.libr_id = 3 and o.type = 'выдача'
group by a.f_name, a.m_name, a.l_name
having count(distinct o.id) >= 2;
```

f_name	m_name	l_name	number_of_reads
Лазарь	Филатович	Данилов	2
Мартьян	Харламович	Александров	2

Рис. 7: Результат запроса 1.2.

Время выполнения: 00:00:00.005652.

Сначала происходит выбор строк из Operation с условиями: reader_id=1, libr_id=3, type='выдача'. Далее последовательно выполняются присоединение таблицы Exemplar по exemplar_id, присоединение таблицы Book по book_id, присоединение таблицы Authorship по book_id и присоединение таблицы Author по

auth_id. Далее происходит группировка записей по Author.f_name, Author.m_name, Author.l_name и фильтрация полученных групп с условием COUNT(DISTINCT Operation.id) >= 2. Представление последавательности выполнения запроса отображено на рисунке 8.

```
Group Key: a.f_name, a.m_name, a.l_name
Filter: (count(DISTINCT o.id) >= 2)
-> Sort  (cost=25.48..25.49 rows=3 width=52)
   Sort Key: a.f_name, a.m_name, a.l_name, o.id
   -> Nested Loop  (cost=1.31..25.46 rows=3 width=52)
      -> Nested Loop  (cost=1.16..24.95 rows=3 width=8)
         Join Filter: (b.id = ash.book_id)
         -> Nested Loop  (cost=0.87..24.56 rows=1 width=12)
            -> Nested Loop  (cost=0.59..24.25 rows=1 width=8)
               -> Index Scan using idx_op_reader_id on "Operation" o  (cost=0.29..15.94 rows=1 width=8)
                  Index Cond: (reader_id = 1)
                  Filter: ((libr_id = 3) AND ((type)::text = 'выдача'::text))
               -> Index Scan using "Exemplar_pkey" on "Exemplar" e  (cost=0.29..8.31 rows=1 width=8)
                  Index Cond: (id = o.exemplar_id)
            -> Index Only Scan using "Book_pkey" on "Book" b  (cost=0.29..0.30 rows=1 width=4)
               Index Cond: (id = e.book_id)
         -> Index Scan using idx_as_book_id on "Authorship" ash  (cost=0.29..0.36 rows=3 width=8)
            Index Cond: (book_id = e.book_id)
      -> Index Scan using "Author_pkey" on "Author" a  (cost=0.15..0.17 rows=1 width=52)
         Index Cond: (id = ash.auth_id)
```

Рис. 8: Explain запроса 1.2.

Реляционная формула:

$$\gamma_{\text{a.f_name,a.m_name,a.l_name,COUNT(DISTINCT o.id)}\rightarrow\text{number_of_reads}}$$
$$(\pi_{\text{a.f_name,a.m_name,a.l_name,o.id}}(\sigma_{\text{o.reader_id=1}\wedge\text{o.libr_id=3}\wedge\text{o.type='выдача'}}(\rho_o(Operation))$$
$$\bowtie_{\text{o.exemplar_id=e.id}} \rho_e(Exemplar) \bowtie_{\text{e.book_id=b.id}} \rho_b(Book)$$
$$\bowtie_{\text{b.id=ash.book_id}} \rho_{ash}(Authorship) \bowtie_{\text{ash.auth_id=a.id}} \rho_a(Author)))_{\text{number_of_reads}\geq 2}$$

4.2.2 Запрос 2

Посчитать число библиотекарей, которые выдавали книги, экземпляры которых были списаны администратором A.

```
select count(distinct o.libr_id) as number_of_librarians
from "Process"."Operation" as o
inner join "Process"."Exemplar" as e on e.id = o.exemplar_id
where e.admin_id = 1 and e.write_off is not null;
```

123 number_of_librarians
99

Рис. 9: Результат запроса 2.

Время выполнения: 00:00:00.177881.

Сначала выбираются записи из таблицы Exemplar по условиям admin_id = 5 и write_off is not null. Далее происходит присоединение таблицы Operation по условию e.id = o.exemplar_id, после чего выполняется устранение дубликатов библиотекарей при помощи оператора distinct и подсчет количества уникальных библиотекарей при помощи count. Представление последавательности выполнения запроса отображено на рисунке 10.

```
-> Sort  (cost=3392.67..3393.61 rows=375 width=4)
   Sort Key: o.libr_id
   -> Nested Loop  (cost=50.83..3376.64 rows=375 width=4)
      -> Bitmap Heap Scan on "Exemplar" e  (cost=50.54..629.86 rows=209 width=4)
         Recheck Cond: (admin_id = 5)
         Filter: (write_off IS NOT NULL)
         -> Bitmap Index Scan on idx_ex_admin_id  (cost=0.00..50.49 rows=4026 width=0)
            Index Cond: (admin_id = 5)
      -> Index Scan using idx_op_exemplar_id on "Operation" o  (cost=0.29..13.11 rows=3 width=8)
         Index Cond: (exemplar_id = e.id)
```

Рис. 10: Explain запроса 2.

Реляционная формула:

$\pi_{\text{COUNT(DISTINCT o.libr_id)}\rightarrow\text{number_of_librarians}}$
$(\gamma_{\text{o.libr_id}}(\pi_{\text{o.libr_id}}(\sigma_{\text{e.admin_id}=5\wedge\text{e.write_off}\neq\text{NULL}}$
$(\rho_e(Exemplar)) \bowtie_{\text{e.id=o.exemplar_id}} \rho_o(Operation))))$

4.2.3 Запрос 3

Для каждого издательства посчитать число экземпляров и число операций, которые с этими экземплярами выполнялись.

```
select p.name, count(distinct e.id)
as number_of_exemplars, count(o.id) as number_of_operations
from "Publication"."Publishing" as p
left join "Publication"."Book" as b on b.publish_id = p.id
left join "Process"."Exemplar" as e on e.book_id = b.id
left join "Process"."Operation" as o on o.exemplar_id = e.id
group by p.name;
```

name	number_of_exemplars	number_of_operations
АО «Емельянов Маркова»	4 260	7 695
АО «Ершова»	3 984	7 232
АО «Красильников-Кудряшов»	3 983	7 188
АО «Лукина, Осипов и Виноградова»	4 089	7 242
АО «Михайлова-Евсеева»	4 093	7 324
АО «Пономарев Туров»	4 169	7 546
Большакова Инкорпорэйтед	4 092	7 289
Евсеев Лимитед	4 041	7 211
ЗАО «Шестакова»	4 015	7 212
ИП «Исакова Афанасьева»	4 367	7 816
ИП «Кузьмин, Чернова и Маслов»	4 044	7 231
НПО «Кабанова»	4 014	7 251
НПО «Суханова-Дорофеева»	4 041	7 240
ОАО «Капустин»	4 381	7 829
ОАО «Соболева Михеева»	4 054	7 216
ОАО «Турова-Ильина»	4 095	7 426
ООО «Лобанова»	4 253	7 646
ООО «Сафонов-Осипова»	3 926	7 058
РАО «Блохин, Суворова и Борисов»	4 130	7 517
РАО «Гордеева, Орехов и Маслов»	4 081	7 299

Рис. 11: Результат запроса 3.

Время выполнения: 00:00:00.964414.

Запрос начинает работу с того, что присоединяет слева таблицу Book к таблице Publishing через publish_id. Далее происходит присоединение слева таблицы Exemplar через book_id и таблицы Operation через exemplar_id. Все присоединение происходят через left join. Далее происходит группировка результатов по названию издательства p.name и подсчет уникальных экземпляров при помощи count и distinct и подсчёт всех операций при помощи count. Представление последавательности выполнения запроса отображено на рисунке 12.

```
Group Key: p.name
-> Sort  (cost=24639.28..25007.95 rows=147468 width=49)
    Sort Key: p.name, e.id
    -> Hash Right Join  (cost=2988.47..6935.62 rows=147468 width=49)
        Hash Cond: (b.publish_id = p.id)
        -> Hash Right Join  (cost=2987.02..6465.04 rows=147468 width=12)
            Hash Cond: (e.book_id = b.id)
            -> Hash Right Join  (cost=2376.52..5467.33 rows=147468 width=12)
                Hash Cond: (o.exemplar_id = e.id)
                -> Seq Scan on "Operation" o  (cost=0.00..2703.68 rows=147468 width=8)
                -> Hash  (cost=1350.12..1350.12 rows=82112 width=8)
                    -> Seq Scan on "Exemplar" e  (cost=0.00..1350.12 rows=82112 width=8)
            -> Hash  (cost=423.00..423.00 rows=15000 width=8)
                -> Seq Scan on "Book" b  (cost=0.00..423.00 rows=15000 width=8)
        -> Hash  (cost=1.20..1.20 rows=20 width=45)
            -> Seq Scan on "Publishing" p  (cost=0.00..1.20 rows=20 width=45)
```

Рис. 12: Explain запроса 3.

Реляционная формула:

$$\gamma_{\text{p.name,COUNT(DISTINCT e.id)}\rightarrow\text{number_of_exemplars,COUNT(o.id)}\rightarrow\text{number_of_operations}}$$

$$(\pi_{\text{p.name,e.id,o.id}}\Big(\rho_p(Publishing) \bowtie_{\text{p.id=b.publish_id}} \rho_b(Book)$$
$$\bowtie_{\text{b.id=e.book_id}} \rho_e(Exemplar) \bowtie_{\text{e.id=o.exemplar_id}} \rho_o(Operation)))$$

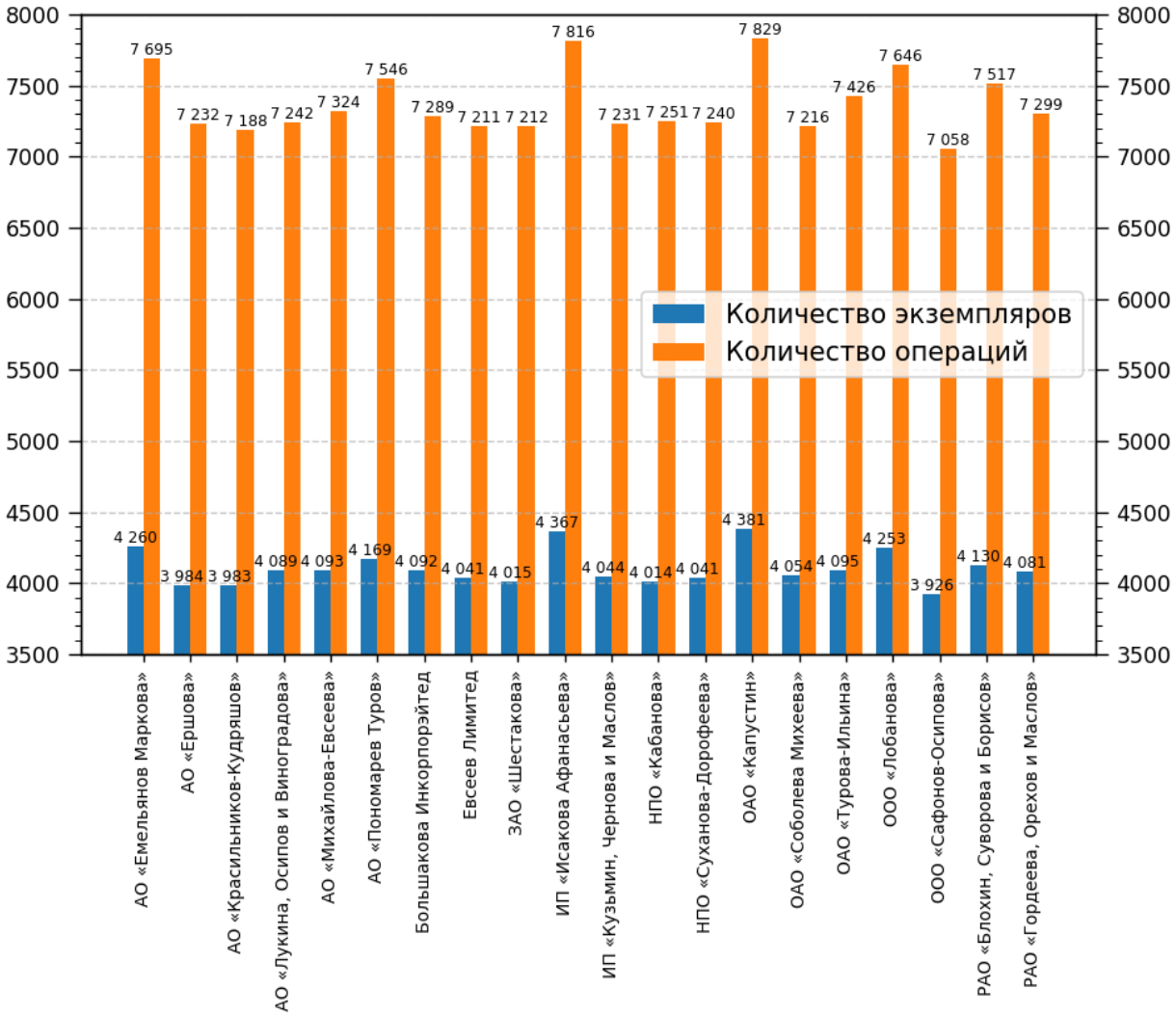


Рис. 13: 2D гистограмма запроса 3.

4.2.4 Запрос 4

Посчитать число книг с одинаковым числом экземпляров

```
select number_of_exemplars, count(book) as number_of_books
from (select b.id as book,  count(e.id) as number_of_exemplars
from"Publication"."Book" as b
left join "Process"."Exemplar" as e on  e.book_id = b.id
group by b.id)
group by number_of_exemplars
order by number_of_exemplars;
```

123 number_of_exemplars	123 number_of_books
1	1 546
2	1 488
3	1 524
4	1 454
5	1 477
6	1 592
7	1 455
8	1 542
9	1 476
10	1 446

Рис. 14: Результат запроса 4

Время выполнения: 00:00:00.045299.

Сначала выполняется внутренний запрос. Таблица Exemplar присоединяется слева к таблице Book по book_id через left join. Далее происходит группировка по Book.id и подсчёт количества связанных экземпляров каждой книги при помощи count. Подзапрос возвращает пары book - id книги и number_of_exemplars - количество экземпляров. Внешний запрос группирует результаты подзапроса по number_of_exemplars, считает количество книг для каждой группы при помощи count и сортирует по возрастанию number_of_exemplars.Представление последавательности выполнения запроса отображено на рисунке 15.

```
Sort  (cost=2945.71..2946.21 rows=200 width=16)
  Sort Key: (count(e.id))
  ->  HashAggregate  (cost=2936.06..2938.06 rows=200 width=16)
        Group Key: count(e.id)
        ->  HashAggregate  (cost=2561.06..2711.06 rows=15000 width=12)
              Group Key: b.id
              ->  Hash Right Join  (cost=584.79..2150.50 rows=82112 width=8)
                    Hash Cond: (e.book_id = b.id)
                    ->  Seq Scan on "Exemplar" e  (cost=0.00..1350.12 rows=82112 width=8)
                    ->  Hash  (cost=397.29..397.29 rows=15000 width=4)
                          ->  Index Only Scan using "Book_pkey" on "Book" b  (cost=0.29..397.29 rows=15000 width=4)
```

Рис. 15: Explain запроса 4.

Реляционная формула:

$$\tau_{\text{number_of_exemplars}}\big(\gamma_{\text{number_of_exemplars},\text{COUNT(book)}\to\text{number_of_books}}$$
$$\big(\gamma_{\text{b.id}\to\text{book},\text{COUNT(e.id)}\to\text{number_of_exemplars}}$$
$$(\pi_{\text{b.id,e.id}}\big(\rho_b(Book) \bowtie_{\text{b.id=e.book_id}} \rho_e(Exemplar)))))$$

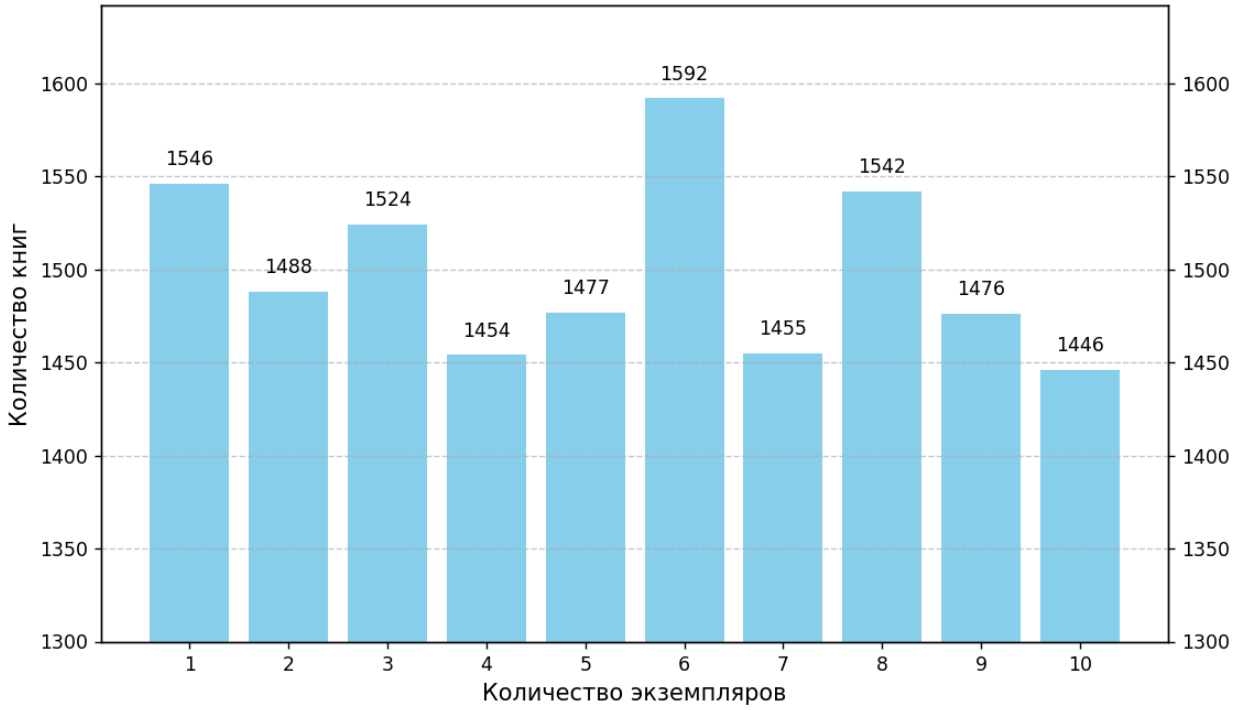


Рис. 16: 2D гистограмма запроса 4.

4.2.5 Запрос 5

Найти библиотекаря с максимальным числом выданных экземпляров и с минимальным числом выданных экземпляров.

```
select l.f_name, l.m_name, l.l_name, count(o.id) as number_of_operations
from "Person"."Librarian" as l
inner join "Process"."Operation" as o on o.libr_id = l.id
where o.type = 'выдача'
group by  l.f_name, l.m_name, l.l_name
having count(o.id) = (select max(cnt) from (select count(id) as cnt
from "Process"."Operation" where type = 'выдача' group by libr_id))
or count(o.id) = (select min(cnt) from (select count(id) as cnt
from "Process"."Operation" where type = 'выдача' group by libr_id))
order by number_of_operations desc;
```

f_name	m_name	l_name	number_of_operations
Варвара	Егоровна	Боброва	820
Тамара	Вячеславовна	Комиссарова	657

Рис. 17: Результат запроса 5.

Время выполнения: 00:00:00.19111.

Сначала выбираются все записи, в которых type = 'выдача' из таблицы Operation. Кполученной таблице присоединяется таблица Librarian по libr_id, после чего данные в таблице группируются по Librarian.f_name, Librarian.m_name и Librarian.l_name. Для каждой группы подсчитывается количество операций через count(o.id). После этого подзапросы вычисляют максимальное и минимальное количество операций среди всех библиотекарей, и having фильтрует только тех библиотекарей, у которых количество операций равно максимуму или минимуму.Итоговые данные сортируются по убыванию количества операций. Представление последавательности выполнения запроса отображено на рисунке 18.

```
Sort  (cost=10788.21..10788.21 rows=1 width=56)
  Sort Key: (count(o.id)) DESC
  InitPlan 1
    ->  Aggregate  (cost=3447.62..3447.63 rows=1 width=8)
          ->  HashAggregate  (cost=3445.37..3446.37 rows=100 width=12)
                Group Key: "Operation".libr_id
                ->  Seq Scan on "Operation"  (cost=0.00..3072.35 rows=74604 width=8)
                      Filter: ((type)::text = 'выдача'::text)
  InitPlan 2
    ->  Aggregate  (cost=3447.62..3447.63 rows=1 width=8)
          ->  HashAggregate  (cost=3445.37..3446.37 rows=100 width=12)
                Group Key: "Operation_1".libr_id
                ->  Seq Scan on "Operation" "Operation_1"  (cost=0.00..3072.35 rows=74604 width=8)
                      Filter: ((type)::text = 'выдача'::text)
  ->  Finalize GroupAggregate  (cost=3880.34..3892.94 rows=1 width=56)
        Group Key: l.f_name, l.m_name, l.l_name
        Filter: ((count(o.id) = (InitPlan 1).col1) OR (count(o.id) = (InitPlan 2).col1))
        ->  Gather Merge  (cost=3880.34..3890.69 rows=90 width=56)
              Workers Planned: 1
              ->  Sort  (cost=2880.33..2880.55 rows=90 width=56)
                    Sort Key: l.f_name, l.m_name, l.l_name
                    ->  Partial HashAggregate  (cost=2876.51..2877.41 rows=90 width=56)
                          Group Key: l.f_name, l.m_name, l.l_name
                          ->  Hash Join  (cost=4.25..2437.66 rows=43885 width=52)
                                Hash Cond: (o.libr_id = l.id)
                                ->  Parallel Seq Scan on "Operation" o  (cost=0.00..2313.32 rows=43885 width=8)
                                      Filter: ((type)::text = 'выдача'::text)
                                ->  Hash  (cost=3.00..3.00 rows=100 width=52)
                                      ->  Seq Scan on "Librarian" l  (cost=0.00..3.00 rows=100 width=52)
```

Рис. 18: Explain запроса 5.

Реляционная формула:

$$\tau_{\text{number_of_operations DESC}}\big(\sigma_{\text{count(o.id)=max_cnt}\vee\text{count(o.id)=min_cnt}}$$
$$\big(\gamma_{\text{l.f_name,l.m_name,l.l_name,count(o.id)}\to\text{number_of_operations}}\big(\pi_{\text{l.f_name,l.m_name,l.l_name,o.id}}$$
$$(\rho_l(\text{Librarian}) \bowtie_{\text{l.id=o.libr_id}} \sigma_{\text{o.type='выдача'}}(\rho_o(\text{Operation}))))))$$

где: $\text{max_cnt} = \gamma_{\text{max(count(id))}\to\text{max_cnt}}\big(\gamma_{\text{libr_id,count(id)}\to\text{cnt}}\big(\sigma_{\text{type='выдача'}}(\text{Operation})\big)$

$\text{min_cnt} = \gamma_{\text{min(count(id))}\to\text{min_cnt}}\big(\gamma_{\text{libr_id,count(id)}\to\text{cnt}}\big(\sigma_{\text{type='выдача'}}(\text{Operation})\big)$

4.2.6 Запрос 6

Найти администраторов, котрые списали книг автора А больше чем администратор Б.

```
select a.f_name, a.m_name, a.l_name,
count(e.id) as number_of_write_offs
from "Process"."Exemplar" as e
inner join "Publication"."Book" as b on e.book_id = b.id
inner join "Publication"."Authorship" as ash on b.id = ash.book_id
inner join "Person"."Administrator" as a on e.admin_id = a.id
where ash.auth_id = 1 and e.write_off is not null
group by a.f_name, a.m_name, a.l_name
having count(e.id) > (
select count(e2.id)
from "Process"."Exemplar" as e2
inner join "Publication"."Book" as b2 on e2.book_id = b2.id
inner join "Publication"."Authorship" ash2 ON b2.id = ash2.book_id
where ash2.auth_id = 1 and e2.write_off is not null and e2.admin_id = 2)
order by number_of_write_offs desc;
```

f_name	m_name	l_name	number_of_write_offs
Спартак	Харлампьевич	Галкин	17
Оксана	Вениаминовна	Колесникова	16
Олимпий	Дорофеевич	Белозеров	16

Рис. 19: Результат запроса 6.

Время выполнения: 00:00:00.039795.

Вначале выполнения запроса таблица экземпляров Exemplar соединяется с таблицей книг Book по book_id, после чего к полученной таблице присоединяется таблица авторов Authorship по book_id и таблица администраторов Administrator по admin_id. выбираются записи, удовлетворяющие условиям auth_id = 1 write_off is not null. Далее все записи группируются по пировка по амилии, имени и отчеству администратора Administrator.f_name, Administrator.m_name и Administrator.l_name, происходит подсчёт количества списанных экземпляров через (count(Exemplar.id)). Следующим шагом подзапрос вычисляет количество списанных экземпляров книг автора с auth_id=1 для администратора с admin_id=2. Having фильтрует только тех администраторов, у которых это количество больше. Результаты сортируются по убыванию количества списанных экземпляров. Представление последавательности выполнения запроса отображено на рисунке 20.

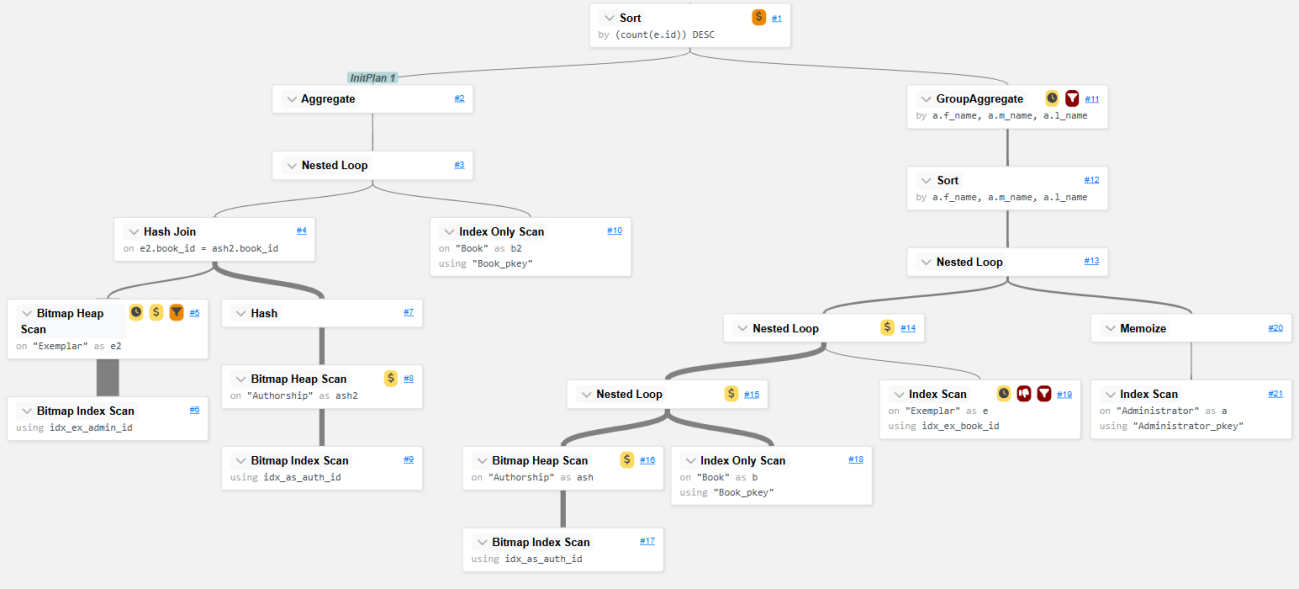


Рис. 20: Графический Explain запроса 6.

Реляционная формула:

$$\tau_{\text{number_of_write_offs DESC}}(\sigma_{\text{count(e.id)>admin2_count}}$$
$$(\gamma_{\text{a.f_name,a.m_name,a.l_name,count(e.id)}\rightarrow\text{number_of_write_offs}}(\pi_{\text{a.f_name,a.m_name,a.l_name,e.id}}$$
$$(\rho_e(\text{Exemplar}) \bowtie_{\text{e.book_id=b.id}} \rho_b(\text{Book})$$
$$\bowtie_{\text{b.id=ash.book_id}} \rho_{ash}(\text{Authorship}) \bowtie_{\text{e.admin_id=a.id}} \rho_a(\text{Administrator}))_{\substack{\text{ash.auth_id=1} \\ \text{e.write_off}\neq\text{NULL}}})))$$
$$\text{где admin2_count} = \gamma_{\text{count(e2.id)}\rightarrow\text{admin2_count}}(\pi_{\text{e2.id}}$$
$$(\rho_{e2}(\text{Exemplar}) \bowtie_{\text{e2.book_id=b2.id}} \rho_{b2}(\text{Book})$$
$$\bowtie_{\text{b2.id=ash2.book_id}} \rho_{ash2}(\text{Authorship}))_{\substack{\text{ash2.auth_id=1} \\ \text{e2.write_off}\neq\text{NULL} \\ \text{e2.admin_id=2}}})$$

4.2.7 Запрос 7

Найти читателей, которые не брали книг издательства A.

```
select r.f_name, r.m_name, r.l_name
from "Person"."Reader" as r
where not exists (select 1 from "Process"."Operation" as o
inner join "Process"."Exemplar" as e on o.exemplar_id = e.id
inner join "Publication"."Book" as b on e.book_id = b.id
where b.publish_id = 4 and o.reader_id = r.id);
```

f_name	m_name	l_name
Вера	Аркадьевна	Козлова

Рис. 21: Результат запроса 7.

Время выполнения: 00:00:00.097193.

Основной запрос выбирает читателей из таблицы Reader, для которых выполняется условие not exists. Подзапрос в not exists присоединяет к таблице Operation таблицу Exemplar по exemplar_id, после чего присоединяет к полученной таблице таблицу Book по book_id. Записи подзапрос фильтрует по publish_id = 4, чтобы получить книги конкретного издательства, и reader_id = Reader.id, чтобы получить операции текущего читателя. Представление последавательности выполнения запроса отображено на рисунке 22.

```
Hash Right Anti Join  (cost=329.20..3857.35 rows=1 width=49)
  Hash Cond: (o.reader_id = r.id)
  ->  Nested Loop  (cost=305.95..3733.53 rows=7314 width=4)
        ->  Hash Join  (cost=305.65..1871.37 rows=4073 width=4)
              Hash Cond: (e.book_id = b.id)
              ->  Seq Scan on "Exemplar" e  (cost=0.00..1350.12 rows=82112 width=8)
              ->  Hash  (cost=296.35..296.35 rows=744 width=4)
                    ->  Bitmap Heap Scan on "Book" b  (cost=14.05..296.35 rows=744 width=4)
                          Recheck Cond: (publish_id = 4)
                          ->  Bitmap Index Scan on idx_bk_publish_id  (cost=0.00..13.87 rows=744
                                Index Cond: (publish_id = 4)
        ->  Index Scan using idx_op_exemplar_id on "Operation" o  (cost=0.29..0.43 rows=3
              Index Cond: (exemplar_id = e.id)
  ->  Hash  (cost=17.00..17.00 rows=500 width=53)
        ->  Seq Scan on "Reader" r  (cost=0.00..17.00 rows=500 width=53)
```

Рис. 22: Explain запроса 7.

Реляционная формула:

$\pi_{\text{r.f_name,r.m_name,r.l_name}}(\sigma_{\text{NOT EXISTS subq}}(\rho_r(\text{Reader})))$

где $\text{subq} = \pi_1(\rho_o(\text{Operation}) \bowtie_{\text{o.exemplar_id=e.id}}$

$\rho_e(\text{Exemplar}) \bowtie_{\text{e.book_id=b.id}} \rho_b(\text{Book}))_{\text{b.publish_id=4} \wedge \text{o.reader_id=r.id}}$

4.2.8 Запрос 8

Для каждого Администратора и каждого издательства подсчитать число списанных экземпляров.

```
select a.f_name, a.m_name, a.l_name, p.name as publishing_name,
count(e.id) filter (where e.write_off is not null)
as number_of_write_offs
FROM "Person"."Administrator" as a
cross join "Publication"."Publishing" as p
left join "Publication"."Book" b on p.id = b.publish_id
left join "Process"."Exemplar" as e
on b.id = e.book_id and e.admin_id = a.id
group by a.f_name, a.m_name, a.l_name, p.name;
```

f_name	m_name	l_name	publishing_name	number_of_write_offs
Марфа	Борисовна	Колобова	ОАО «Соболева Михеева»	14
Ульян	Герасимович	Шаров	Евсеев Лимитед	17
Аникита	Егорович	Никонов	ОАО «Соболева Михеева»	12
Варлаам	Адрианович	Киселев	АО «Ершова»	10
Анжелика	Владимировна	Щукина	Евсеев Лимитед	4
Георгий	Игоревич	Карпов	ЗАО «Шестакова»	11
Анжелика	Владимировна	Щукина	ОАО «Соболева Михеева»	16
Спартак	Харлампьевич	Галкин	НПО «Суханова-Дорофеева»	12
Аникита	Егорович	Никонов	РАО «Гордеева, Орехов и Маслов»	5
Олимпиада	Георгиевна	Кошелева	ОАО «Капустин»	15
Анжелика	Владимировна	Щукина	НПО «Суханова-Дорофеева»	8
Спартак	Харлампьевич	Галкин	РАО «Гордеева, Орехов и Маслов»	10

Рис. 23: Результат запроса 8.

Время выполнения: 00:00:00.776724.

В начале выполнения запроса создаются все возможные пары администраторов из таблицы Administrator и издательств из таблицы Publishing при помощи cross join. Далее к каждой паре администратор-издательство присоединяются книги из таблицы Book по publish_id через left join, а к книгам присоединяюися экземпляры из Exemplar через book_id с дополнительным условием Exemplar.admin_id = Admin.id также через left join. После этого происходит группировка по фамилии, имени и отчеству администратора Administrator.f_name, Administrator.m_name и Administrator.l_name и названию издательства Publishing.name. В конце запрос подсчитывает количество списанных экземпляров через count(Exemplar.id) с фильтром Exemplar.write_off is not null. Представление последавательности выполнения запроса отображено на рисунке 24.

```
HashAggregate  (cost=29500.82..29504.82 rows=400 width=100)
  Group Key: a.f_name, a.m_name, a.l_name, p.name
  ->  Hash Right Join  (cost=13411.42..25750.82 rows=300000 width=104)
        Hash Cond: ((e.book_id = b.id) AND (e.admin_id = a.id))
        ->  Seq Scan on "Exemplar" e  (cost=0.00..1350.12 rows=82112 width=20)
        ->  Hash  (cost=4223.42..4223.42 rows=300000 width=100)
              ->  Nested Loop  (cost=1.45..4223.42 rows=300000 width=100)
                    ->  Hash Right Join  (cost=1.45..472.17 rows=15000 width=45)
                          Hash Cond: (b.publish_id = p.id)
                          ->  Seq Scan on "Book" b  (cost=0.00..423.00 rows=15000 width=8)
                          ->  Hash  (cost=1.20..1.20 rows=20 width=45)
                                ->  Seq Scan on "Publishing" p  (cost=0.00..1.20 rows=20 width=45)
                    ->  Materialize  (cost=0.00..1.30 rows=20 width=55)
                          ->  Seq Scan on "Administrator" a  (cost=0.00..1.20 rows=20 width=55)
```

Рис. 24: Explain запроса 8.

Реляционная формула:

$$\gamma_{\text{a.f_name,a.m_name,a.l_name,p.name,COUNT(e.id) FILTER (e.write_off IS NOT NULL)}\rightarrow\text{number_of_write_offs}}$$
$$(\pi_{\text{a.f_name,a.m_name,a.l_name,p.name,e.id}}$$
$$(\rho_a(\text{Administrator}) \times \rho_p(\text{Publishing})\bowtie_{\text{p.id=b.publish_id}}\rho_b(\text{Book})$$
$$\bowtie_{\substack{\text{b.id=e.book_id}\\ \text{e.admin_id=a.id}}} \rho_e(\text{Exemplar})))$$

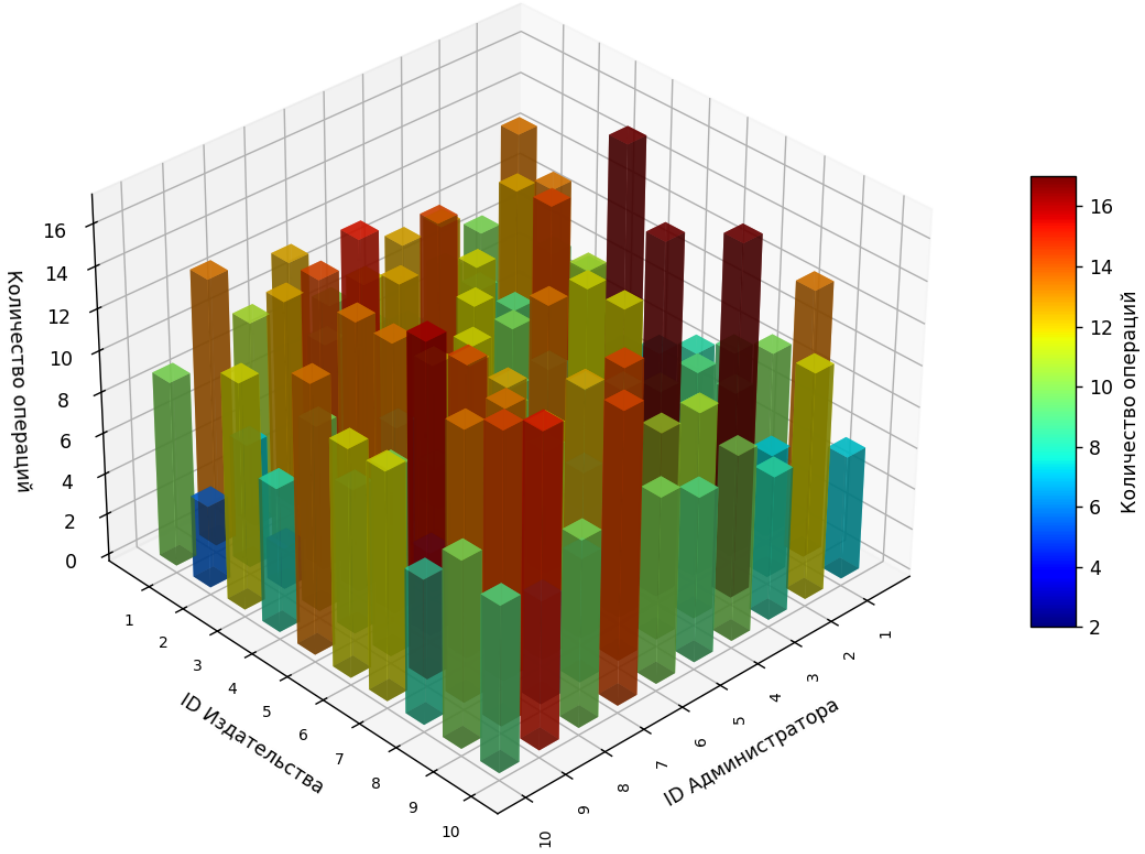


Рис. 24: 3D гистограмма запроса 8.

Заключение

В ходе курсовой работы был выполнен анализ предметной области, по результатам которого были разработаны ER-диаграмма, включающая в себя 11 сущностей, схема связей между объектами, содержащая 8 объектов, и две схемы структуры базы данных на русском и английском языках. На основании этого была реализована база данных в СУБД Postgres, включающая в себя 9 таблиц. Для заполнения базы данных была написана программа на языке Python, при помощи которой было сгенерировано 290 293 записей. Также в ходе программирования было выполнено 8 SQL-запросов к базе данных, один из которых был реализована в двух вариантах. Было построено две 2D гисторaммы и одна 3D гистограмма.

Курсовая работа была выполнена на устройсте Asus Vivobook, с прцессором intel core i5 и оперативной памятью 16 гигабайт. Также использовались диалект PostgreSql, DBeaver 24.3.0, командная строка, Visual Studio версии 17.8.3 и Python версии 3.13.

Использованные источники

1. Кузнецов С. Д. Базы данных. Модели и языки / Кузнецов С. Д. - Москва: Бином Пресс. 2008.

2. Попов С. Г. Лист самостоятельной проверки хода выполнения курсовой работы по дисциплине «Теоретические основы баз данных» 4 семестр.

3. - Библиотека [Электронный ресурс]: Википедия. URL: https://ru.wikipedia.org/wiki/Библиотека (Дата обращения: 10.04.2025)

4. Российская национальная библиотека, СПБ. URL: https://nlr.ru/ (Дата обращения: 11.04.2025)

5. Ю. Н. Столяров - Библиотеки [Электронный ресурс]: Большая российская энциклопедия. URL: https://old.bigenc.ru/text/5061145 (Дата обращения: 9.04.2025)

Приложение А: Создание базы данных

```
create schema "Person";
create schema "Publication";
create schema "Process";
create table "Person"."Librarian"(
id serial primary key,
f_name varchar(1478) not null,
m_name varchar(14) not null,
l_name varchar(700) not null,
pass_ser int4 not null,
pass_num int4 not null,
phone varchar(12) not null,
constraint check_passser check
    (pass_ser > 999 and pass_ser < 10000),
constraint check_passnum check
    (pass_num > 99999 and pass_num < 1000000),
constraint unique_pass unique (pass_ser, pass_num),
constraint unique_phone unique (phone),
constraint check_phone check (phone ~ '^\+\d{11}$'),
constraint check_fname check (f_name ~ '^[A-Za-zA-Яа-яЁё -]+$'),
constraint check_mname check (m_name ~ '^[A-Za-zA-Яа-яЁё -]+$'),
constraint check_lname check (l_name ~ '^[A-Za-zA-Яа-яЁё -]+$')
);
create table "Person"."Administrator"(
id serial primary key,
f_name varchar(1478) not null,
m_name varchar(14) not null,
l_name varchar(700) not null,
pass_ser int4 not null,
pass_num int4 not null,
phone varchar(12) not null,
constraint check_apassser check
    (pass_ser > 999 and pass_ser < 10000),
constraint check_apassnum check
    (pass_num > 99999 and pass_num < 1000000),
constraint unique_apass unique (pass_ser, pass_num),
constraint unique_aphone unique (phone),
constraint check_aphone check (phone ~ '^\+\d{11}$'),
```

```
constraint check_afname check (f_name ~ '^[A-Za-zA-Яа-яЁё -]+$'),
constraint check_amname check (m_name ~ '^[A-Za-zA-Яа-яЁё -]+$'),
constraint check_alname check (l_name ~ '^[A-Za-zA-Яа-яЁё -]+$')
);
create table "Person"."Reader"(
id serial primary key,
f_name varchar(1478) not null,
m_name varchar(14) not null,
l_name varchar(700) not null,
pass_ser int4 not null,
pass_num int4 not null,
phone varchar(12) not null,
address varchar(512) not null,
reg_date date not null constraint default_rd default now(),
libr_id int4 not null,
constraint fk_librid foreign key(libr_id)
references "Person"."Librarian"(id)
on update no action,
constraint check_rpassser check
    (pass_ser > 999 and pass_ser < 10000),
constraint check_rpassnum check
    (pass_num > 99999 and pass_num < 1000000),
constraint unique_rpass unique (pass_ser, pass_num),
constraint unique_rphone unique (phone),
constraint check_rphone check (phone ~ '^\+[0-9]{11}$'),
constraint check_rfname check (f_name ~ '^[A-Za-zA-Яа-яЁё -]+$'),
constraint check_rmname check (m_name ~ '^[A-Za-zA-Яа-яЁё -]+$'),
constraint check_rlname check (l_name ~ '^[A-Za-zA-Яа-яЁё -]+$')
);
create table "Publication"."Author"(
id serial primary key,
f_name varchar(1478) not null,
m_name varchar(14) not null,
l_name varchar(700) not null,
constraint check_aufname check (f_name ~ '^[A-Za-zA-Яа-яЁё -]+$'),
constraint check_aumname check (m_name ~ '^[A-Za-zA-Яа-яЁё -]+$'),
constraint check_aulname check (l_name ~ '^[A-Za-zA-Яа-яЁё -]+$')
);
create table "Publication"."Publishing"(
```

```
id serial primary key,
name varchar(50) not null,
phone varchar(12) not null,
address varchar(512) not null,
constraint unique_padress unique (address),
constraint unique_pphone unique (phone),
constraint check_pphone check (phone ~ '^\+\d{11}$'),
constraint check_pname check (name ~
    '^[A-Za-zA-Яа-яЁё0-9\s''"«».,:()\---]+$')
);
create table "Publication"."Book"(
id serial primary key,
ISBN char(17) not null,
name varchar(65535) not null,
publish_date date not null,
UDK varchar(15) not null,
publish_id int4 not null,
constraint unique_ISBN unique (ISBN),
constraint check_pdate check (publish_date <= now()),
constraint fk_publid foreign key(publish_id)
references "Publication"."Publishing"(id)
on update no action,
constraint check_bname check
            (name ~ '^[A-Za-zA-Яа-яЁё0-9\s''".,:()\---]+$'),
constraint check_ISBN check
            (ISBN ~'^(978|979)-[0-9]-[0-9]{5}-[0-9]{3}-[0-9]$'),
constraint check_UDK check (UDK~'^[0-9]{3}.[0-9.]+$')
);
create table "Publication"."Authorship" (
id serial primary key,
book_id int4 not null,
auth_id int4 not null,
constraint fk_bookid foreign key(book_id)
references "Publication"."Book"(id)
on update no action,
constraint fk_authid foreign key(auth_id)
references "Publication"."Author"(id)
on update no action
);
```

```
create table "Process"."Exemplar" (
id serial primary key,
book_id int4 not null,
admin_id int4 not null,
receipt timestamp not null constraint default_r default now(),
write_off timestamp null constraint default_wo default null,
constraint fk_bookid foreign key(book_id)
references "Publication"."Book"(id)
on update no action,
constraint fk_adminid foreign key(admin_id)
references "Person"."Administrator"(id)
on update no action,
constraint check_rec check (receipt <= now()),
constraint check_wo check (write_off <= now()),
constraint check_date check (write_off IS null or write_off > receipt)
);
create table "Process"."Operation" (
id serial primary key,
reader_id int4 not null,
libr_id int4 not null,
exemplar_id int4 not null,
op_date timestamp not null constraint default_d default now(),
type varchar(10) not null,
constraint fk_librid foreign key(libr_id)
references "Person"."Librarian"(id)
on update no action,
constraint fk_readerid foreign key(reader_id)
references "Person"."Reader"(id)
on update no action,
constraint fk_exemplarid foreign key(exemplar_id)
references "Process"."Exemplar"(id)
on update no action,
constraint unique_operation unique (reader_id, exemplar_id, op_date)
);
create index idx_ex_book_id on "Process"."Exemplar" (book_id);
create index idx_ex_admin_id on "Process"."Exemplar" (admin_id);
create index idx_bk_publish_id on "Publication"."Book" (publish_id);
create index idx_op_reader_id on "Process"."Operation" (reader_id);
create index idx_op_libr_id on "Process"."Operation" (libr_id);
```

```
create index idx_op_exemplar_id on "Process"."Operation" (exemplar_id);
create index idx_rd_libr_id on "Person"."Reader" (libr_id);
create index idx_as_book_id on "Publication"."Authorship" (book_id);
create index idx_as_auth_id on "Publication"."Authorship" (auth_id);
```

Приложение B: Заполнение базы данных

```
import psycopg2
from faker import Faker
from random import randint, choice, sample
import datetime as dt
from datetime import date, datetime
import logging
from collections import defaultdict
import datetime
from datetime import date
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

fake = Faker('ru_RU')
DB_CONFIG = {
    "dbname": "postgres",
    "user": "postgres",
    "password": "Safi_4mic",
    "host": "127.0.0.1",
    "port": 5432
}

NUM_PUBLISHINGS = 20
NUM_AUTHORS = 50
NUM_ADMINS = 20
NUM_LIBRARIANS = 100
NUM_READERS = 500
NUM_BOOKS = 15000
MIN_AUTHORS_PER_BOOK = 1
MAX_AUTHORS_PER_BOOK = 5
MIN_EXEMPLARS_PER_BOOK = 1
MAX_EXEMPLARS_PER_BOOK = 10
MIN_OPERATIONS_PER_READER = 150
MAX_OPERATIONS_PER_READER = 600

def create_connection():
    return psycopg2.connect(**DB_CONFIG)
```

```
def generate_phone():
    return '+7' + ''.join([str(randint(0, 9)) for _ in range(10)])

def generate_person():
    gender = choice(['male', 'female'])
    if gender == 'male':
        return (
            fake.first_name_male(),
            fake.middle_name_male(),
            fake.last_name_male()
        )
    else:
        return (
            fake.first_name_female(),
            fake.middle_name_female(),
            fake.last_name_female()
        )

def fill_librarians(conn):
    with conn.cursor() as cur:
        batch = []
        for _ in range(NUM_LIBRARIANS):
            f_name, m_name, l_name = generate_person()
            batch.append((
                f_name,
                m_name,
                l_name,
                randint(1000, 9999),
                randint(100000, 999999),
                generate_phone()
            ))
            if len(batch) >= 1000:
                cur.executemany("""
                    INSERT INTO "Person"."Librarian"
                    (f_name, m_name, l_name, pass_ser, pass_num, phone)
                    VALUES (%s, %s, %s, %s, %s, %s)
                """, batch)
                batch = []
        if batch:
```

```
            cur.executemany("""
                INSERT INTO "Person"."Librarian"
                (f_name, m_name, l_name, pass_ser, pass_num, phone)
                VALUES (%s, %s, %s, %s, %s, %s)
            """, batch)
        conn.commit()

def fill_administrators(conn):
    with conn.cursor() as cur:
        batch = []
        for _ in range(NUM_ADMINS):
            f_name, m_name, l_name = generate_person()
            batch.append((
                f_name,
                m_name,
                l_name,
                randint(1000, 9999),
                randint(100000, 999999),
                generate_phone()
            ))
            if len(batch) >= 1000:
                cur.executemany("""
                    INSERT INTO "Person"."Administrator"
                    (f_name, m_name, l_name, pass_ser, pass_num, phone)
                    VALUES (%s, %s, %s, %s, %s, %s)
                """, batch)
                batch = []
        if batch:
            cur.executemany("""
                INSERT INTO "Person"."Administrator"
                (f_name, m_name, l_name, pass_ser, pass_num, phone)
                VALUES (%s, %s, %s, %s, %s, %s)
            """, batch)
        conn.commit()

def fill_authors(conn):
    with conn.cursor() as cur:
        batch = []
        for _ in range(NUM_AUTHORS):
```

```
            f_name, m_name, l_name = generate_person()
            batch.append((f_name, m_name, l_name))
            if len(batch) >= 1000:
                cur.executemany("""
                    INSERT INTO "Publication"."Author"
                    (f_name, m_name, l_name)
                    VALUES (%s, %s, %s)
                """, batch)
                batch = []
        if batch:
            cur.executemany("""
                INSERT INTO "Publication"."Author"
                (f_name, m_name, l_name)
                VALUES (%s, %s, %s)
            """, batch)
        conn.commit()

def fill_publishings(conn):
    with conn.cursor() as cur:
        batch = []
        for _ in range(NUM_PUBLISHINGS):
            batch.append((
                fake.company(),
                generate_phone(),
                fake.address()
            ))
            if len(batch) >= 1000:
                cur.executemany("""
                    INSERT INTO "Publication"."Publishing"
                    (name, phone, address)
                    VALUES (%s, %s, %s)
                """, batch)
                batch = []
        if batch:
            cur.executemany("""
                INSERT INTO "Publication"."Publishing"
                (name, phone, address)
                VALUES (%s, %s, %s)
            """, batch)
```

```
        conn.commit()

ddef fill_readers(conn):
    with conn.cursor() as cur:
        cur.execute('SELECT id FROM "Person"."Librarian"')
        librarian_ids = [row[0] for row in cur.fetchall()]

        batch = []
        for _ in range(NUM_READERS):
            f_name, m_name, l_name = generate_person()
            batch.append((
                f_name,
                m_name,
                l_name,
                randint(1000, 9999),
                randint(100000, 999999),
                generate_phone(),
                fake.address(),
                choice(librarian_ids)
            ))
            if len(batch) >= 1000:
                cur.executemany("""
                    INSERT INTO "Person"."Reader"
                    (f_name, m_name, l_name,
                    pass_ser, pass_num, phone, address, libr_id)
                    VALUES (%s, %s, %s, %s, %s, %s, %s, %s)
                """, batch)
                batch = []
        if batch:
            cur.executemany("""
                INSERT INTO "Person"."Reader"
                (f_name, m_name, l_name,
                pass_ser, pass_num, phone, address, libr_id)
                VALUES (%s, %s, %s, %s, %s, %s, %s, %s)
            """, batch)
        conn.commit()

def fill_books(conn):
    with conn.cursor() as cur:
```

```
        cur.execute('SELECT id FROM "Publication"."Publishing"')
        publishing_ids = [row[0] for row in cur.fetchall()]

        batch = []
        for _ in range(NUM_BOOKS):
            prefix = randint(978, 979)
            group = randint(0, 9)
            publisher = f"{randint(0, 99999):05d}"
            title = f"{randint(0, 999):03d}"
            check_digit = randint(0, 9)
            isbn = f"{prefix}-{group}-{publisher}-{title}-{check_digit}"

            publish_date =  \
            fake.date_between(start_date='-30y', end_date='today')
            batch.append((
                isbn,
                fake.catch_phrase(),
                publish_date,
                f"{randint(100, 999)}.{randint(0,9)}.{randint(0,9)}",
                choice(publishing_ids)
            ))
            if len(batch) >= 1000:
                cur.executemany("""
                    INSERT INTO "Publication"."Book"
                    (ISBN, name, publish_date, UDK, publish_id)
                    VALUES (%s, %s, %s, %s, %s)
                """, batch)
                batch = []

        if batch:
            cur.executemany("""
                INSERT INTO "Publication"."Book"
                (ISBN, name, publish_date, UDK, publish_id)
                VALUES (%s, %s, %s, %s, %s)
            """, batch)
        conn.commit()

def fill_authorship(conn):
    with conn.cursor() as cur:
```

```
        cur.execute('SELECT id FROM "Publication"."Book"')
        book_ids = [row[0] for row in cur.fetchall()]

        cur.execute('SELECT id FROM "Publication"."Author"')
        author_ids = [row[0] for row in cur.fetchall()]

        batch = []
        for book_id in book_ids:
            num_authors = randint(MIN_AUTHORS_PER_BOOK, \
            MAX_AUTHORS_PER_BOOK)
            selected_authors = sample(author_ids, num_authors)

            for auth_id in selected_authors:
                batch.append((book_id, auth_id))
                if len(batch) >= 1000:
                    cur.executemany("""
                        INSERT INTO "Publication"."Authorship"
                        (book_id, auth_id)
                        VALUES (%s, %s)
                    """, batch)
                    batch = []

        if batch:
            cur.executemany("""
                INSERT INTO "Publication"."Authorship"
                (book_id, auth_id)
                VALUES (%s, %s)
            """, batch)
        conn.commit()

def fill_exemplars(conn):
    with conn.cursor() as cur:
        cur.execute('SELECT id, publish_date FROM "Publication"."Book"')
        books = cur.fetchall()

        cur.execute('SELECT id FROM "Person"."Administrator"')
        admin_ids = [row[0] for row in cur.fetchall()]

        batch = []
```

```
for book in books:
    book_id, publish_date = book

    if isinstance(publish_date, dt.date):
        publish_datetime = dt.datetime.combine(publish_date,
        dt.time(0, 0))
    else:
        publish_datetime = publish_date
    exemplar_count = randint(MIN_EXEMPLARS_PER_BOOK,
    MAX_EXEMPLARS_PER_BOOK)

    for _ in range(exemplar_count):
        receipt_date = fake.date_time_between(
            start_date=publish_datetime,
            end_date=dt.datetime.now()
        )

        write_off = None
        if randint(1, 100) > 95:
            max_write_off = min(
                receipt_date + dt.timedelta(days=3*365),
                dt.datetime.now()
            )
            write_off = fake.date_time_between(
                start_date=receipt_date,
                end_date=max_write_off
            )

        batch.append((book_id, choice(admin_ids), \
        receipt_date, write_off))

        if len(batch) >= 1000:
            cur.executemany("""
                INSERT INTO "Process"."Exemplar"
                (book_id, admin_id, receipt, write_off)
                VALUES (%s, %s, %s, %s)
            """, batch)
            batch = []
```

```
        if batch:
            cur.executemany("""
                INSERT INTO "Process"."Exemplar"
                (book_id, admin_id, receipt, write_off)
                VALUES (%s, %s, %s, %s)
            """, batch)
        conn.commit()

def fill_operations(conn):
    with conn.cursor() as cur:
        cur.execute('SELECT id FROM "Person"."Reader"')
        reader_ids = [row[0] for row in cur.fetchall()]
        cur.execute('SELECT id FROM "Person"."Librarian"')
        librarian_ids = [row[0] for row in cur.fetchall()]
        cur.execute('SELECT id FROM "Process"."Exemplar"')
        exemplar_ids = [row[0] for row in cur.fetchall()]
        batch = []
        for reader_id in reader_ids:
            num_operations = randint(MIN_OPERATIONS_PER_READER,
            MAX_OPERATIONS_PER_READER)
            op_dates = [
                fake.date_time_between(start_date='-1y', end_date='now')
                for _ in range(num_operations)
            ]
            op_dates.sort()
            for op_date in op_dates:
                exemplar_id = choice(exemplar_ids)
                op_type = choice(['выдача', 'возврат'])
                libr_id = choice(librarian_ids)
                batch.append((
                    reader_id,
                    libr_id,
                    exemplar_id,
                    op_date,
                    op_type
                ))
                if len(batch) >= 1000:
                    cur.executemany("""
                        INSERT INTO "Process"."Operation"
```

```
                        (reader_id, libr_id, exemplar_id, op_date, type)
                        VALUES (%s, %s, %s, %s, %s)
                    """, batch)
                    batch = []
        if batch:
            cur.executemany("""
                INSERT INTO "Process"."Operation"
                (reader_id, libr_id, exemplar_id, op_date, type)
                VALUES (%s, %s, %s, %s, %s)
            """, batch)
        conn.commit()

if __name__ == "__main__":
    conn = create_connection()
    try:
        logger.info("Starting data generation...")

        fill_librarians(conn)
        fill_administrators(conn)
        fill_publishings(conn)
        fill_authors(conn)
        fill_readers(conn)
        fill_books(conn)
        fill_authorship(conn)
        fill_exemplars(conn)
        fill_operations(conn)

        logger.info("Data generation completed successfully!")
    except Exception as e:
        logger.error(f"Error: {str(e)}")
        conn.rollback()
    finally:
        conn.close()
```